Linear Regression

Fit a straight line. Predict a number.

$$y = mx + b$$

y prediction · **m** slope · **x** input · **b** intercept

Vitor Machado

Finding the line

Goal: pick m (slope) and b (intercept) that minimize the sum of squared errors.

$$\underbrace{m}_{\text{slope}} = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{n}(x_i - \bar{x})^2} \qquad \underbrace{b}_{\text{intercept}} = \bar{y} - m\bar{x}$$

Slope = covariance(x, y) / variance(x) — how steeply the line rises.

Intercept = where the line crosses the y-axis; anchors it through the means $(\bar{x}, \bar{y})$.

Worked Example

Step 1 — compute the means ($\bar{x}$, $\bar{y}$ are the averages of x and y):

$$\bar{x} = (50 + 70 + 80 + 100 + 120)/5 = 84 \qquad \bar{y} = (150 + 200 + 230 + 300 + 350)/5 = 246$$

Step 2 — build the sums:

	x_i	y_i	$x_i - \bar{x}$	$y_i - \bar{y}$	$(x_i - \bar{x})(y_i - \bar{y})$	$(x_i - \bar{x})^2$
	50	150	−34	−96	3,264	1,156
	70	200	−14	−46	644	196
	80	230	−4	−16	64	16
	100	300	16	54	864	256
	120	350	36	104	3,744	1,296
Σ					**8,580**	**2,920**

Step 3 — plug into the formulas:

$$m = 8{,}580 \,/\, 2{,}920 \approx 2.94 \qquad b = 246 - 2.94 \times 84 \approx -0.82$$

Evaluating the fit

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2$$

Average squared error. Smaller is better.

$$R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}$$

$R^2 = 1.0$ is perfect, 0 means no better than predicting the mean.

Live Demo

Watch the line fit in real time

Pros

- **Interpretable by design** — each coefficient has a literal meaning; required in regulated domains
- **Trains in milliseconds** — closed-form solution, no GPU, millions of rows on a laptop
- **Cheap to serve** — model is a handful of floats; inference is one dot product
- **Statistical inference for free** — p-values, confidence intervals, standard errors per coefficient
- **The baseline everything must beat** — if a complex model barely wins, the complexity isn't justified

Cons

- **Assumes a linear relationship** — if the truth curves, the bias is structural; no amount of data fixes it
- **Outliers dominate the fit** — squared loss lets one extreme point pull the entire line
- **Multicollinearity breaks interpretation** — correlated features make coefficients unstable and flip signs
- **Extrapolation fails silently** — predicts confidently far outside the training range, with no warning
- **Inference has strings attached** — p-values and CIs assume independent, normal, homoscedastic errors

Use Case — Real Estate Pricing

Problem: predict house price from square meters.

Square Meters	Price (USD)
50	150,000
70	200,000
80	230,000
100	300,000
120	350,000

The model learns $price = b + m \cdot sqm$, then predicts any new size.

Other applications: sales forecasting, crop yield estimation, stock trends.

Code Walkthrough

`LinearRegression.fit` — ~20 lines, no libraries

Takeaway

Always try linear first.

Use R^2 and **MSE** to judge the fit.

Reach for something richer only when the data is clearly non-linear.